ETH zürich

D-INFK Lehre



Statistical machine learning

Carlos Cotrini, Konstantinos Chasiotis, Dimitrios Dekas, Alper Karadeniz, Mattia de Martino

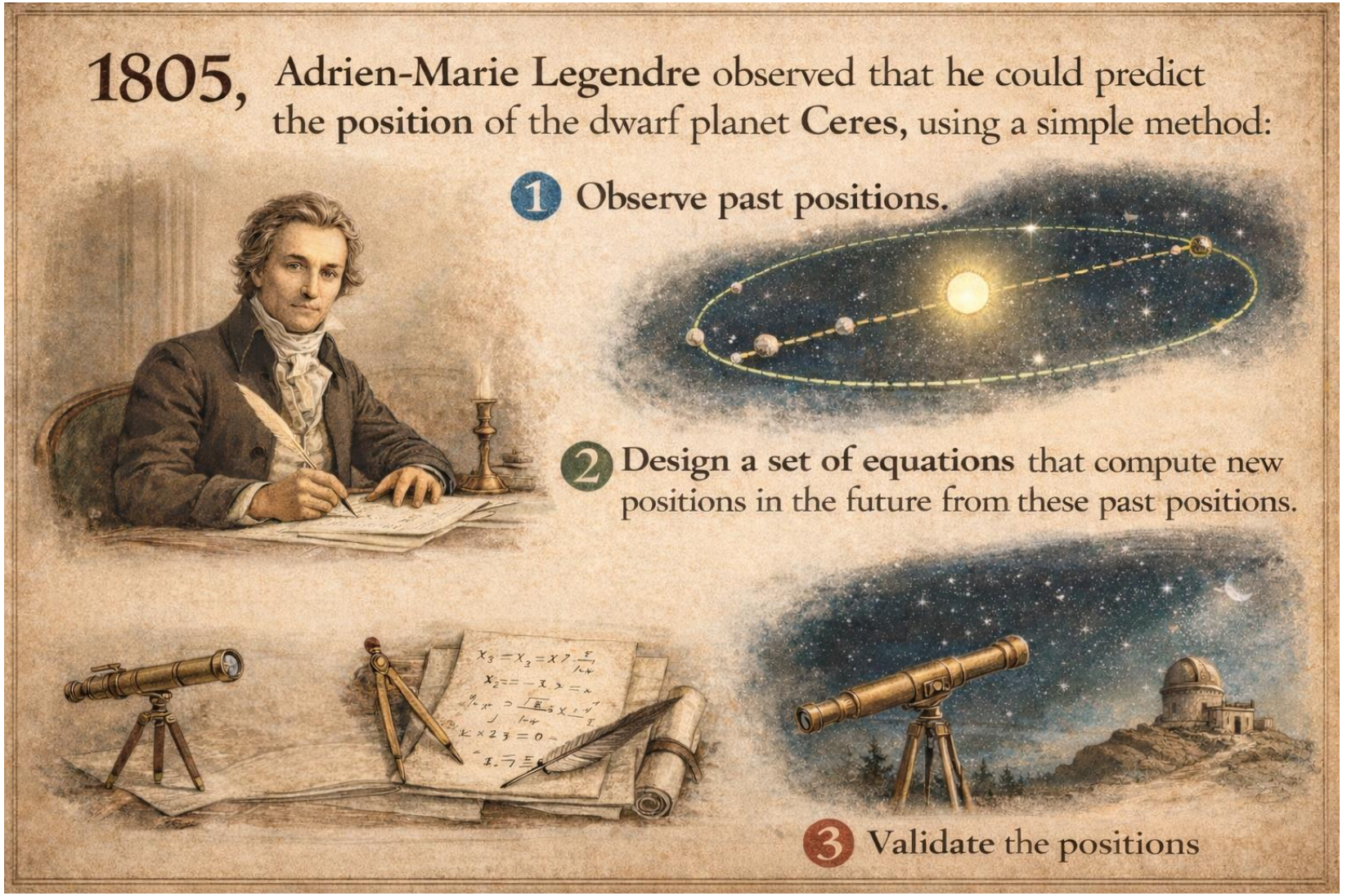
Building ML/AI applications

2026

ETH zürich

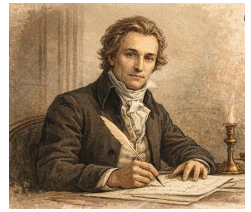
Plan for this weekend

The scientific method

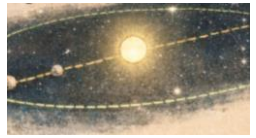


1

Knowledge discovery



Observe



Hypothesize



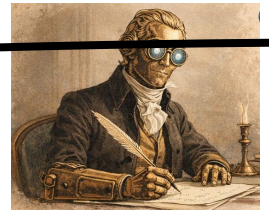
Predict and validate



AI

1

2

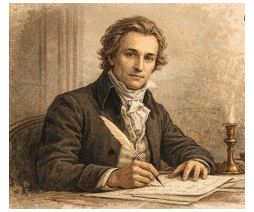


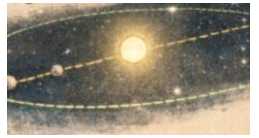
Knowledge discovery

Observe

Hypothesize

Predict and validate









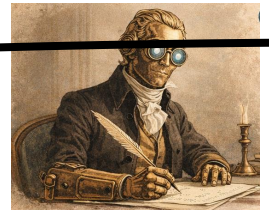
AI

ML

1

2

3

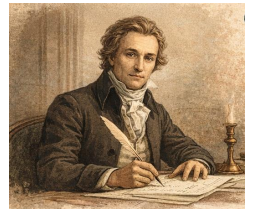


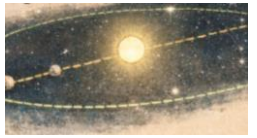
Knowledge discovery

Observe

Hypothesize

Predict and validate









AI

ML

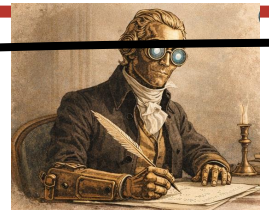
Vapnik's statistical learning

1

2

3

4

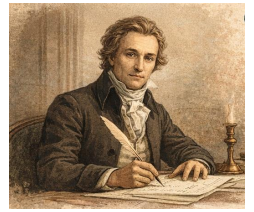


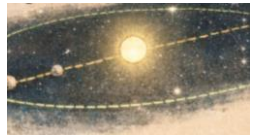
Knowledge discovery

Observe

Hypothesize

Predict and validate









AI

ML

Vapnik's statistical learning

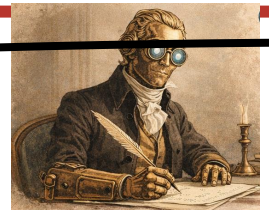
1

2

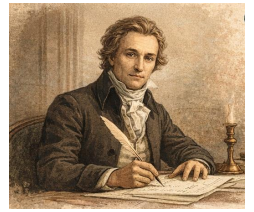
3

4

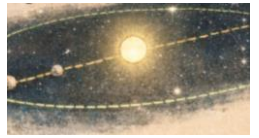
5



Knowledge discovery



Observe



Hypothesize

Loss function

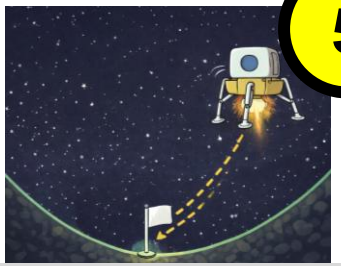
Model / Hypothesis class

Training algorithm



Predict and validate





Gradient descent as the training algorithm

AI

ML

Vapnik's statistical learning

1

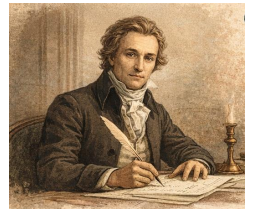
2

3

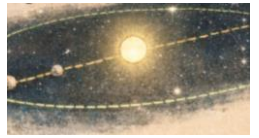
4



Knowledge discovery



Observe



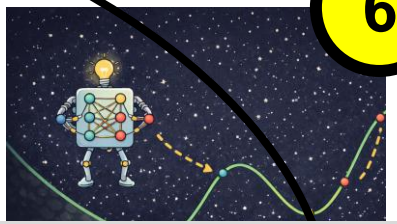
Hypothesize

Loss function

Model / Hypothesis class

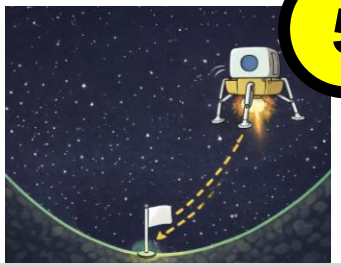
Training algorithm

6



Neural networks as models

5



Gradient descent as the training algorithm

Predict and validate





AI

ML

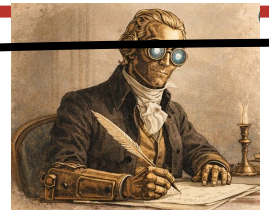
Vapnik's statistical learning

1

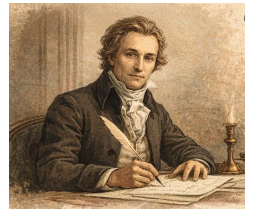
2

3

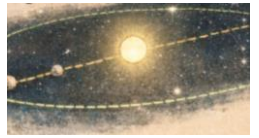
4



Knowledge discovery



Observe



Hypothesize

Loss function

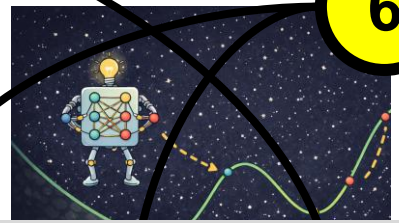
Model / Hypothesis class

Training algorithm

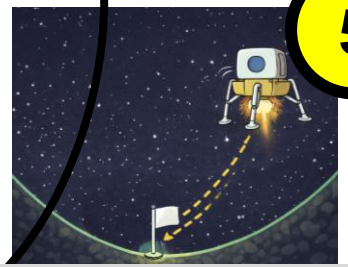


Predict and validate





Neural networks as models



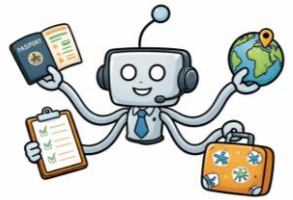
Gradient descent as the training algorithm

5

6

7

8



Mini travel agent

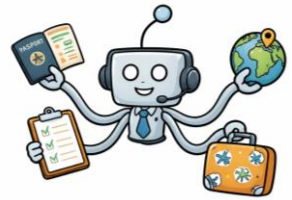


Mini meeting assistant

AI

ML

Vapnik's statistical learning

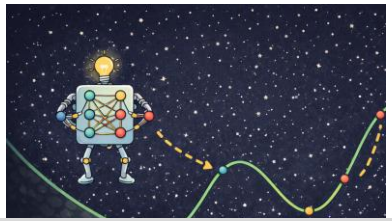


Mini travel agent

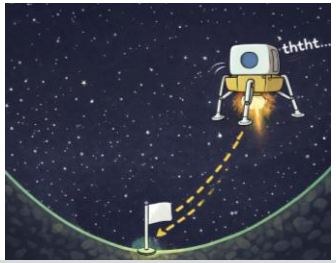


Mini meeting assistant





Neural networks as models



Gradient descent as the training algorithm

Knowledge discovery

Observe

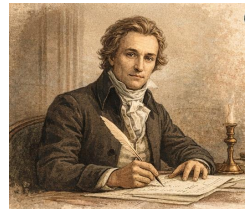
Hypothesize

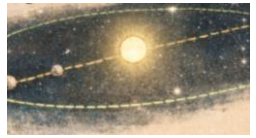
Loss function

Model / Hypothesis class

Training algorithm

Predict and validate







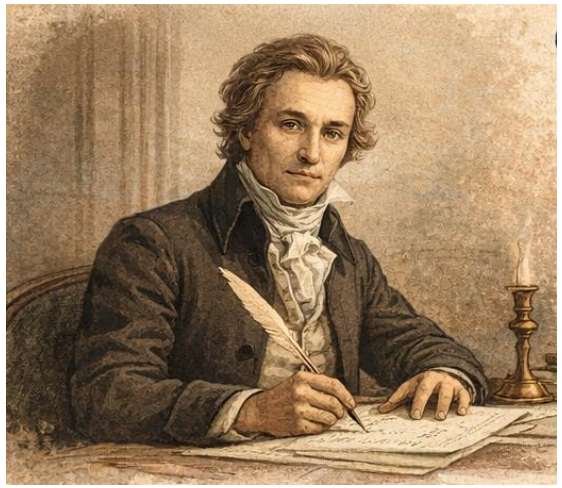


ETH zürich

ETH zürich

(Surprising) dependencies on data

Human learning (the scientific method)



Observe

Hypothesize

Validate







The main goal of machine learning



Observe

Hypothesize

Validate







Side note: Agentic AI follows from this pipeline extended with a fourth step: **Act**

The first ML application in history

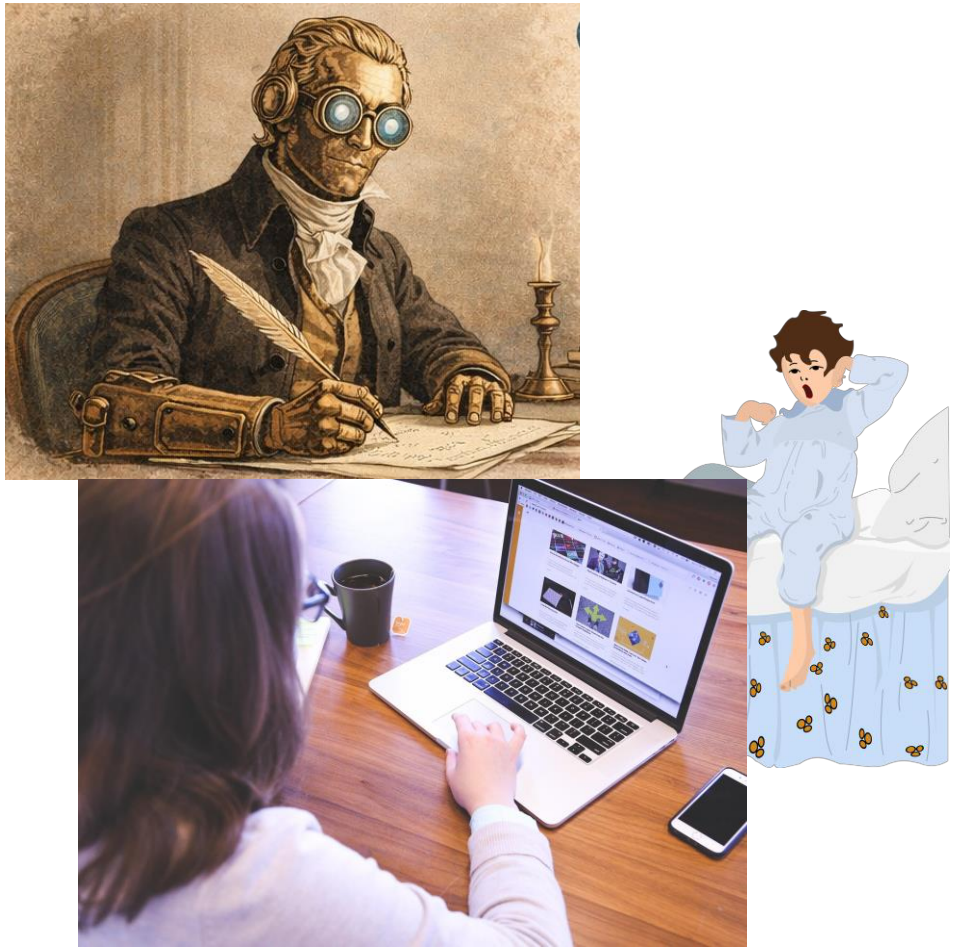
- **J. A. C. Brown, H. S. Houthakker, and S. J. Prais (1953)**, *"Electronic Computation in Economic Statistics,"* **Journal of the American Statistical Association**, 48(263): 414–428.

- A computer was used to discover an equation that would explain how families spend money on specific goods based on certain measurements like income, age, location, etc…



A leap forward: the machine discovers trackers on its own

- Predict CDC flu surveillance data weeks in advance.
- A basic model appeared to "understand" epidemiology.
- It **discovered on its own** search terms that people type in search engines when they are about to get the flu.
- With Google search logs, institutions can anticipate flu trends at a faster pace than waiting for hospital records.



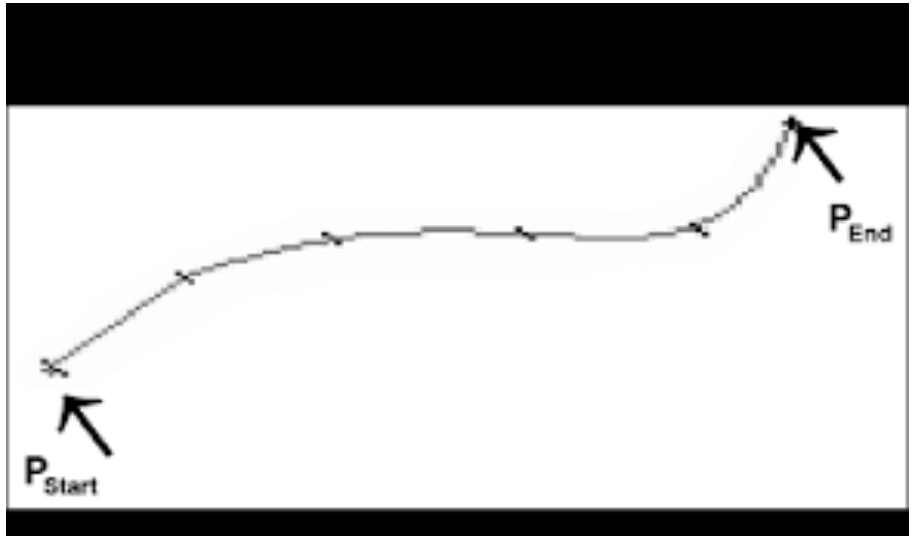
Going beyond: diagnosing Alzheimer before visible symptoms appear



- A ML algorithm that tells if a person has Parkinson disease (PD) based on their interactions with a website.

Paper: https://onlinelibrary.wiley.com/doi/full/10.1002/acn3.50945

Going beyond: diagnosing Alzheimer before visible symptoms appear



- Mouse pointer's location.
- Keyboard features.
  - Number of typos.
  - Time between results shown and keyboard use.
- …
- These features are not related to Parkinson nor to Bing.

So what is an ML application?

Any computer application that consumes data…



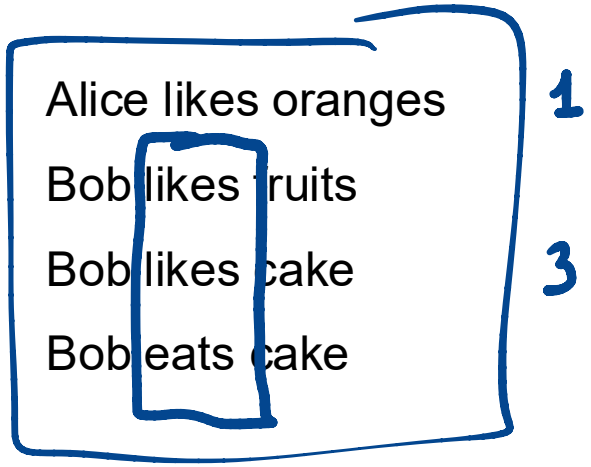
…computes a **model** from that data…

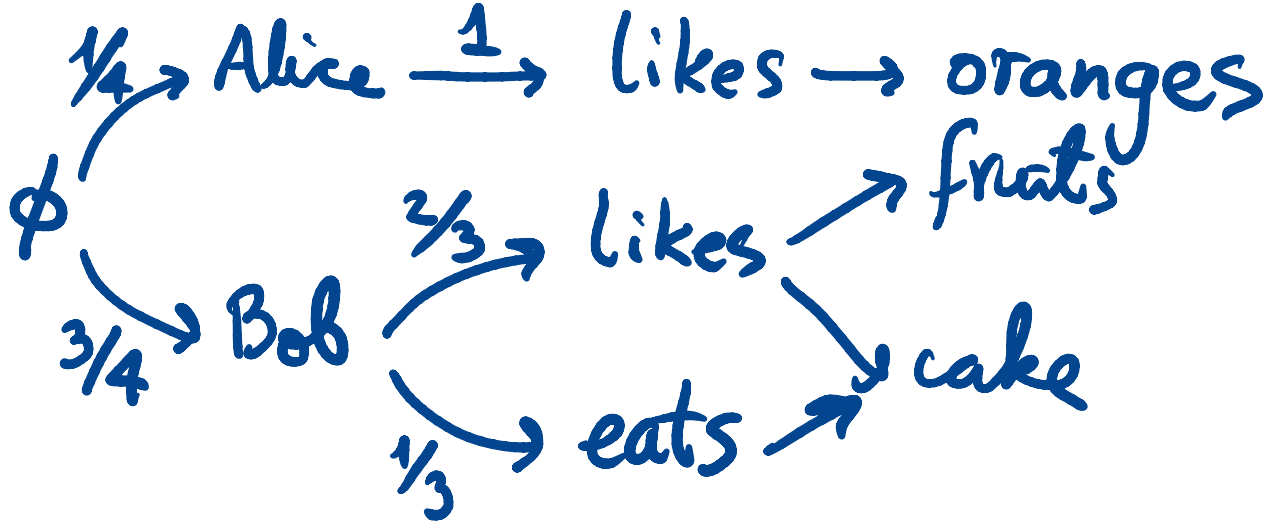


…makes **predictions** using that **model.**



More modern applications: (large) language models





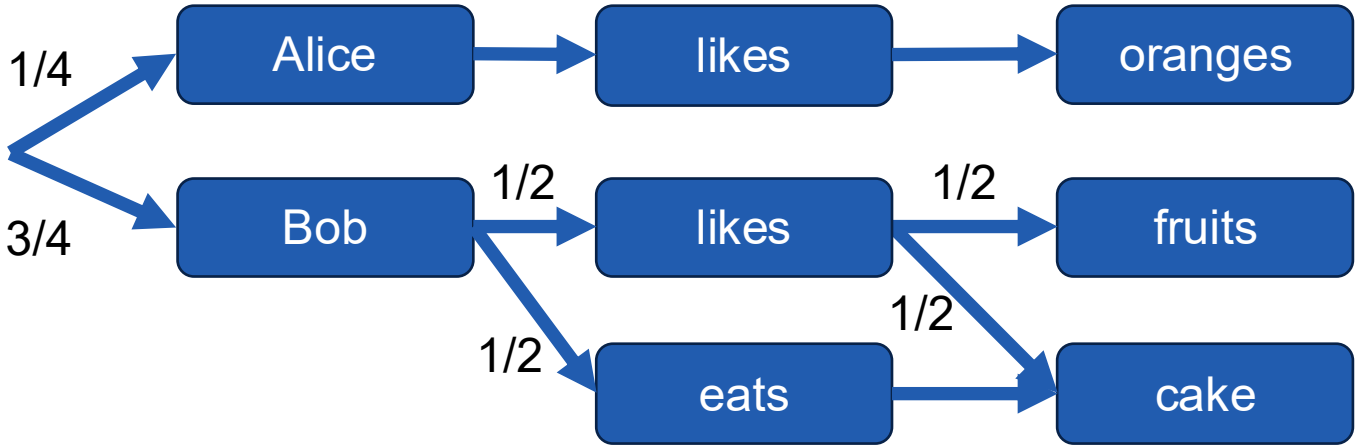
More modern applications: (large) language models

Alice likes oranges

Bob likes fruits

Bob likes cake

Bob eats cake



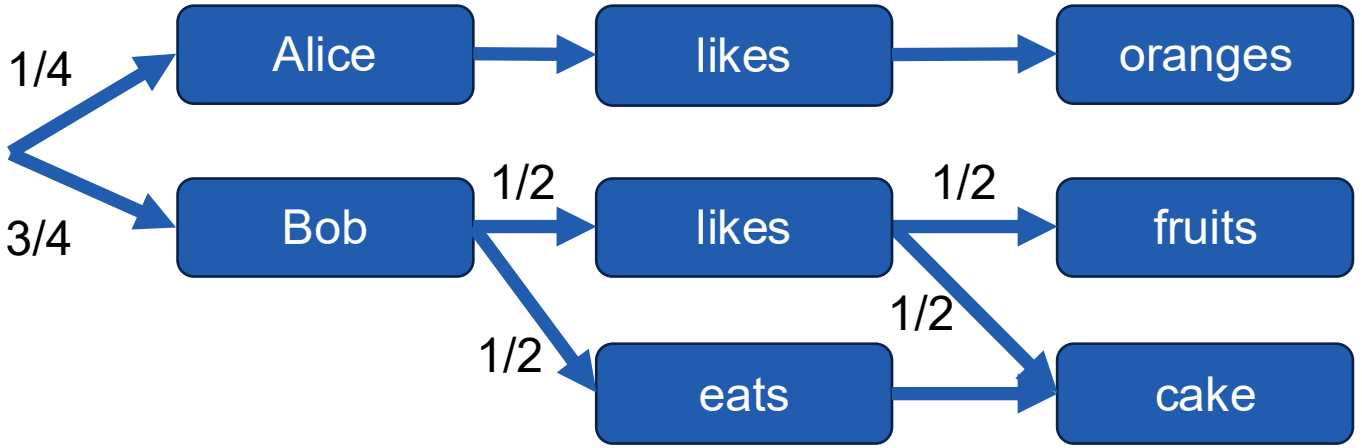
More modern applications: (large) language models

Alice likes oranges

Bob likes fruits

Bob likes cake

Bob eats cake



A	B	L	E	C	F	O
0	0	0	0	0	0	0
1	0	0	0	0	0	0
1	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
0	1	0	1	0	0	0

A	B	L	E	C	F	O
1/4	3/4	0	0	0	0	0
0	0	1	0	0	0	0
0	0	0	0	0	0	1
0	0	1/2	1/2	0	0	0
0	0	0	0	1/2	1/2	0
0	0	0	1	0	0	0

ML Classification Pipeline

Complete workflow from raw data to trained model



1 Understand & Visualize

- **Import** dataset
- **Explore:** `head()` `info()` `describe()`
- Check if data is **imbalanced**
- **Visualize:** histograms, pairplots

2 Prepare Data

- **Split:** train / val / test
- **Standardize** with `StandardScaler()`
- Fit on **X_train only**
- Transform all sets with same scaler

3 Select Model

- **Logistic Reg** (sigmoid → 0-1)
- **Multiclass** (softmax for K classes)
- **KNN** (K-nearest neighbors)
- **Decision Tree** (bagging, bootstrap)

4 Train & Optimize

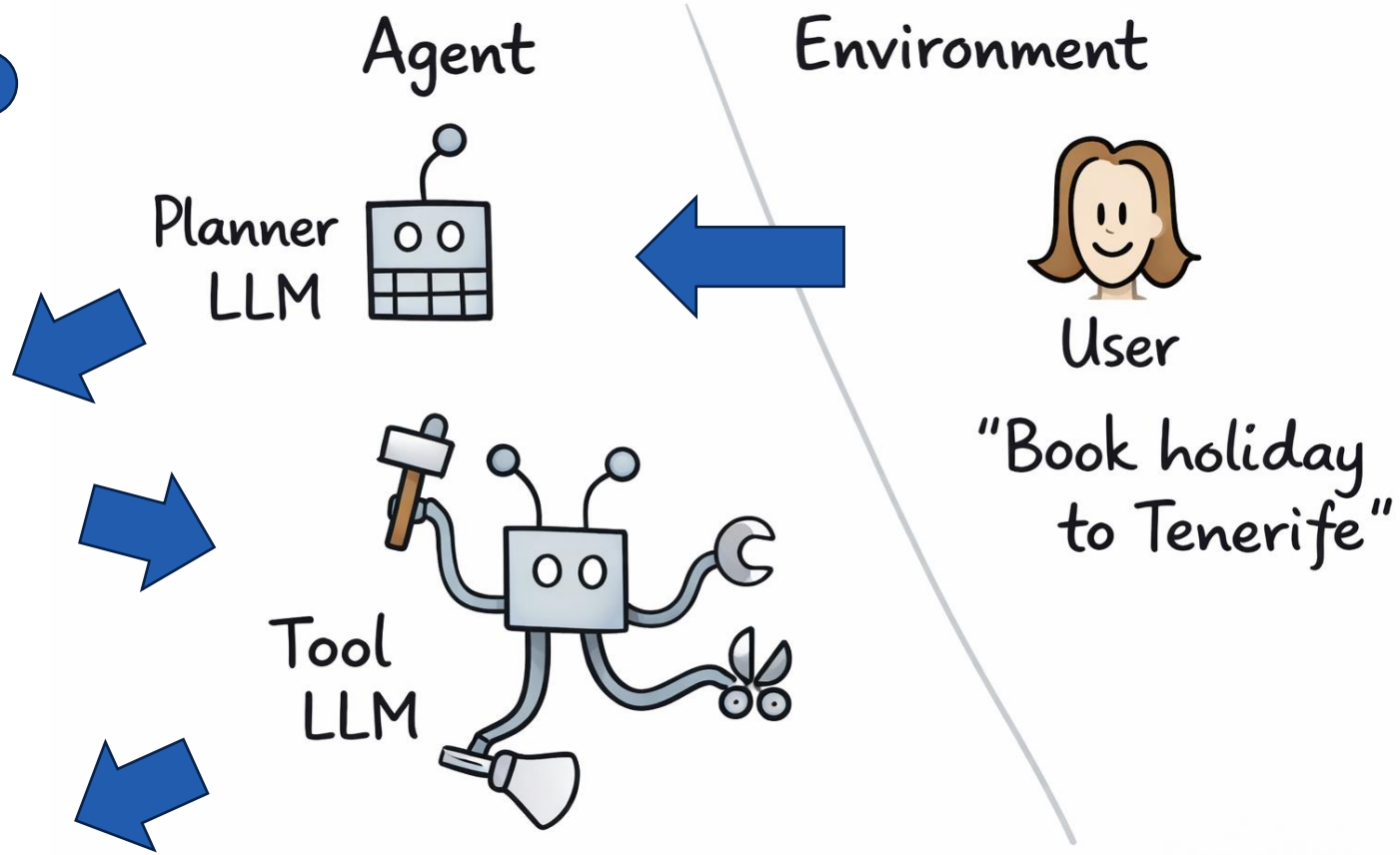
- **Fit** model on training data
- **Loss:** Binary / Categorical Cross Entropy
- **Optimize** via gradient descent

5 Evaluate Performance

- **Evaluate** model on validation data
- **Balanced Accuracy**
- **Confusion Matrix**
- **Precision / Recall / F1**
- **ROC-AUC / PR-AUC**

What about agentic AI?

I need to **book flights** to Tenerife**.** Then I need to **organize car rental.** Finally, I need to **book hotel.**



Book flights

Car rental

Book hotel

Agentic AI requires to decompose a plan into concrete actions and those actions need to be translated into **syntactically correct calls** to **specialized computer applications.** This translation is a specialized ML application.

Vapnik’s statistical learning theory

AI

ML

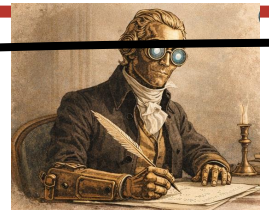
Vapnik's statistical learning

1

2

3

4



Knowledge discovery

Observe

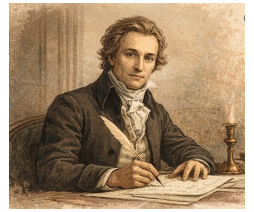
Hypothesize

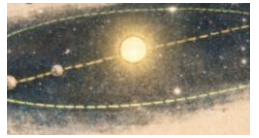
Loss function

Model / Hypothesis class

Training algorithm

Predict and validate

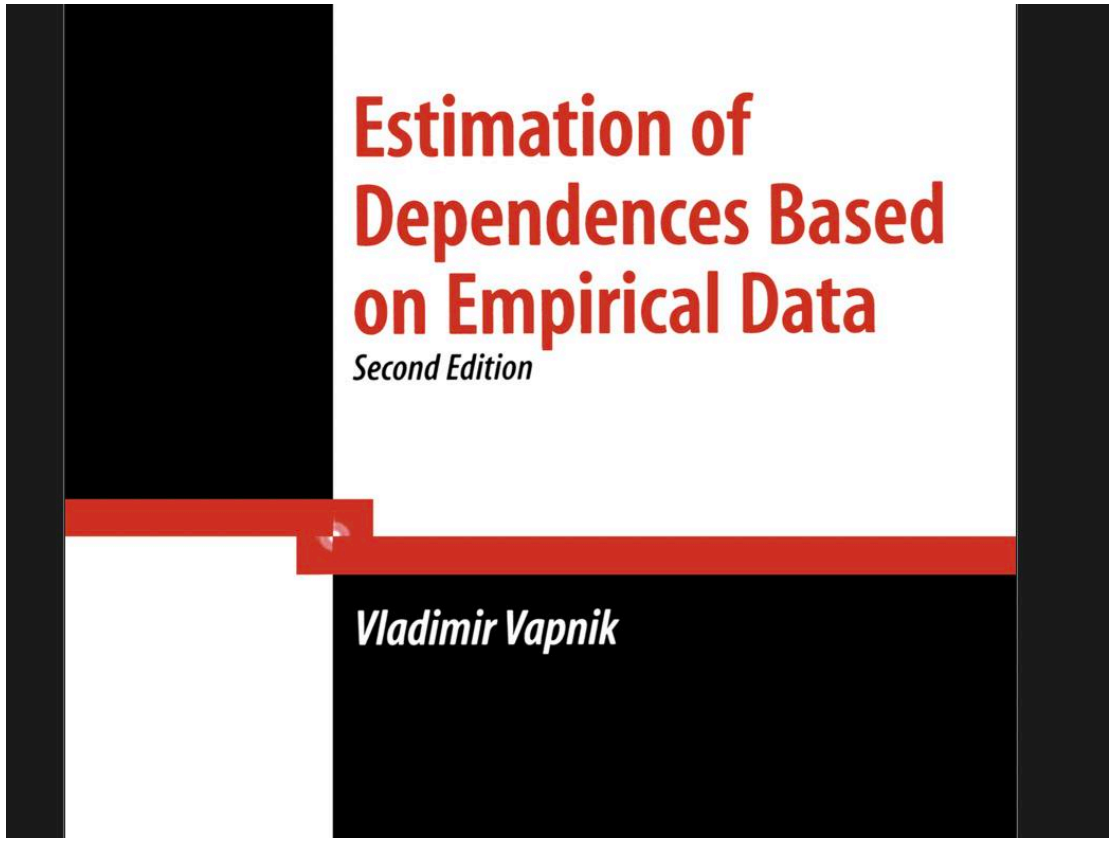








ML as dependence estimation



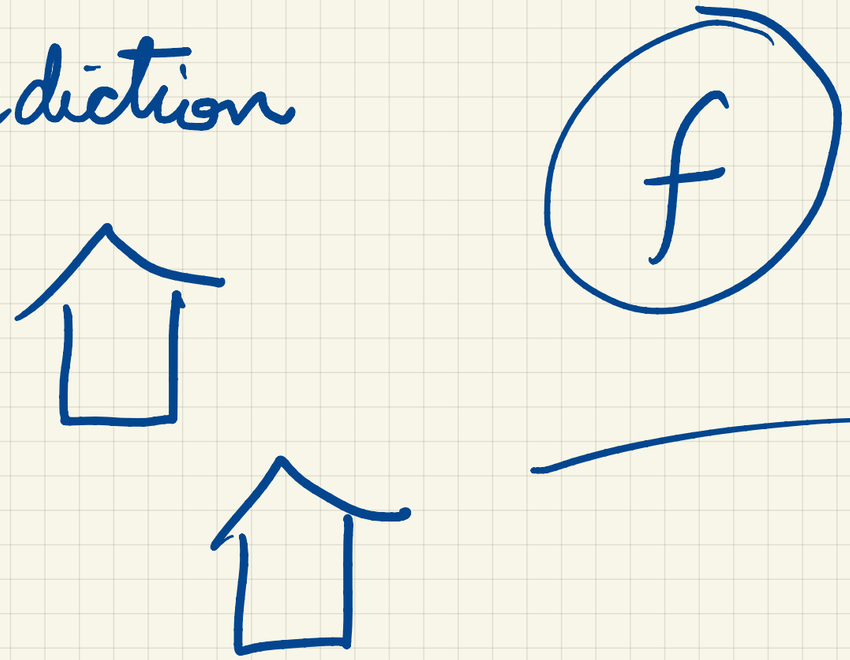
- In 1982, Vladimir Vapnik established **empirical risk minimization:** the main strategy behind many modern ML applications.

Estimating dependences on the basis of empirical data has been, and will probably remain, a central problem in applied analysis. This problem is a mathematical interpretation of one of the basic questions of science: how to extract the existing law-like relationship from scattered data.

The simplest attack on this problem is to construct (estimate) a function from its values at certain points. Here we will formulate some general principles of estimating a functional dependence, and then develop an algorithm for the estimation using these principles.

Usually, when one seeks a general principle, intended for a solution of a wide class of problems, one focuses first upon the simplest, most basic problem. This simple version of the problem is treated theoretically with great thoroughness and the scheme obtained for a solution is then extended to all the problems of the class under consideration.

House prediction



1) Observe $(\text{⌂}, \$)$ $(\text{⌂}, \$\$) \dots (\text{⌂}, \$\$\$)$

x_1, y_1 x_2, y_2 x_n, y_n

2) Hypothesize $y = wx + b$

← later replace w/ sthe better

3) Predict & validate

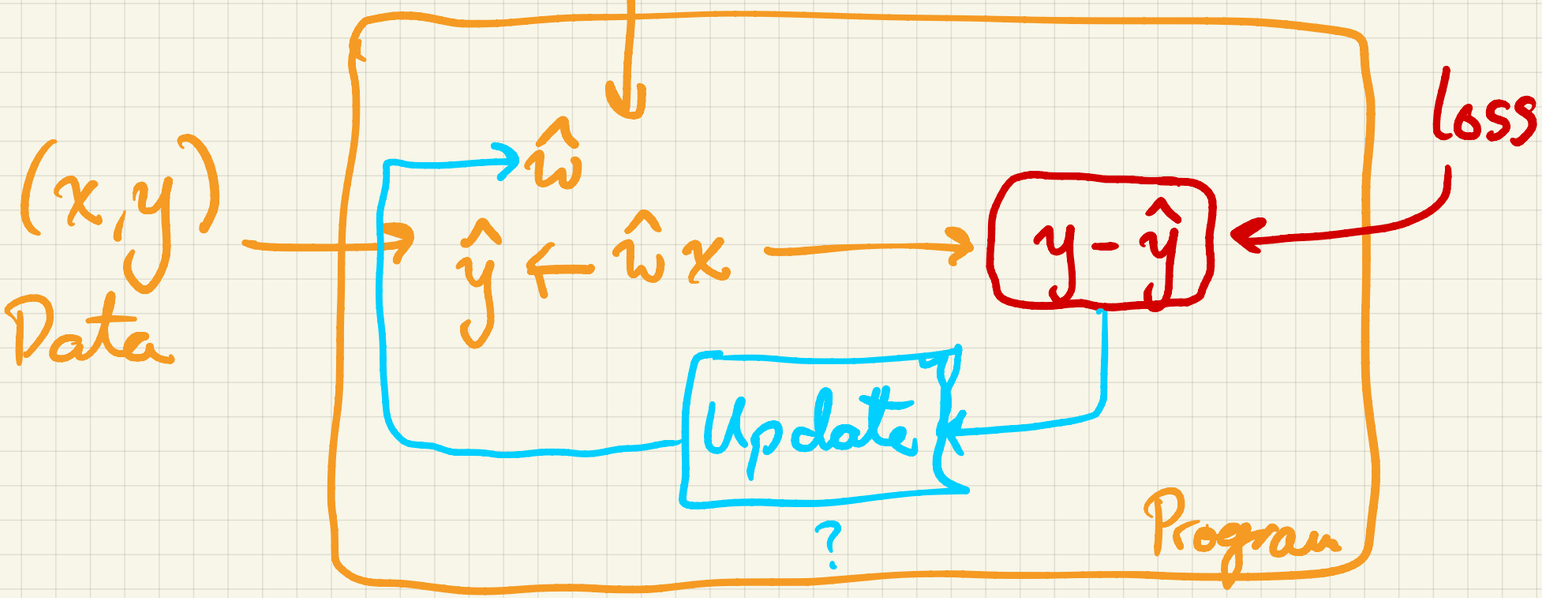
w: ? b: ?

First step

Given $(x, y) \in \mathbb{R} \times \mathbb{R}$

Assume $y = wx$ ← unknown

1st algo



$w_3 = 849.5$

???

How do we estimate w?

$x = 234$

$y = 198,789$

$w_0 = 900$

$\hat{y}_0 = w_0 x =$ 210 600

$w_1 = 350$

$\hat{y}_1 = w_1 x = 81\,900$

$w_2 = 850$

$\hat{y}_2 =$ 198,900

Fixed-step algorithm

Given x, y

Guess $\hat{w} \leftarrow \$$

do

$\hat{y} = \hat{w} x$

$\ell = y - \hat{y}$

if $\ell < 0$ then $\hat{w} \leftarrow \hat{w} - \alpha$

else $\hat{w} \leftarrow \hat{w} + \alpha$

until $|\ell| < \varepsilon$

you choose

Robbins-Monro update

$$\hat{w} \leftarrow \hat{w} + \alpha \ell$$

Multi-dim RM

$y = wx + b$

↑ area ↑ base

Given (x, y)

Guess $\hat{w}$, $\hat{b}$

do

$\hat{y} = \hat{w}x + \hat{b}$

$l = y - \hat{y}$

update $\hat{w}$ based on l

$\hat{y} = \hat{w}x + \hat{b}$

$l = y - \hat{y}$

update $\hat{b}$ based on l

until $|l| < \varepsilon$

Batch MD RM algo

Given $(x_1, y_1) \dots (x_n, y_n)$

Guess for $\hat{w}, \hat{b}$

do

for $j \le n$: $\hat{y}_j = \hat{w} x_j + \hat{b}$

$l_j = y_j - \hat{y}_j$

$l = \frac{1}{n} \sum_{j \le n} l_j$

update ...

until $|l| < \varepsilon$

$x = 234$

$y = 198789$

$w_0 = 850 \rightarrow \hat{y} = 190.000$

$y - \hat{y} = 8789$

Multi-variate model

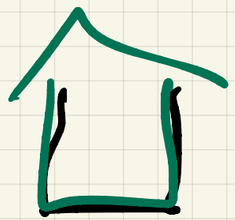
$(x_1, y_1) \dots (x_n, y_n)$

Guess $\hat{w}, \hat{b}$

do

- compute predictions
- compute losses
- update some w_j

until $|l| < \varepsilon$



$x_i \in \mathbb{R}^d$

$x_i = (x_{i1}, x_{i2}, \dots, x_{id})$

area, #rooms

$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id} + b$

$= w^T x_i + b$

$w = (w_1, w_2, \dots, w_d)$

RM algorithm $$

Given (x_1, y_1) (x_2, y_2) ... (x_n, y_n)

$(x_{11}, x_{12}, x_{13} \dots x_{1d})$

Guess $\hat{w}, \hat{b}$

$\hat{w} = (\hat{w}_1, \hat{w}_2, \hat{w}_3, \dots, \hat{w}_d)$

Model: $\hat{y} = \hat{w}^T x + \hat{b}$

do

for $j \leq d$:

for $i \leq n$:

$\hat{y}_i \leftarrow \hat{w}^T x_i + \hat{b}$

$l_i = y_i - \hat{y}_i$

$l = \frac{1}{n} \sum_i l_i$

compute loss l

$\hat{w}_j \leftarrow \hat{w}_j + \alpha l$

until $|l| < \varepsilon$

About loss functions

$l = y - \hat{y}$

$l = |y - \hat{y}|$

$l = (y - \hat{y})^2$

$l = (y - \hat{y})^3$

$l = (y - \hat{y})^4$

Huber loss

$\delta \downarrow$

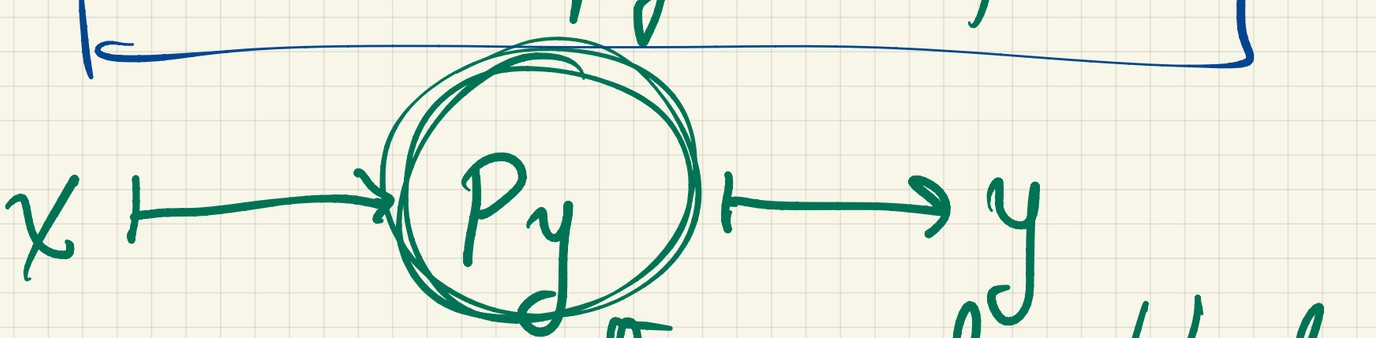
$$\begin{cases} \frac{1}{2}(y-\hat{y})^2 & |y-\hat{y}| \le \delta \\ \delta(|y-\hat{y}| - \frac{1}{2}\delta) & |y-\hat{y}| > \delta \end{cases}$$

Classification

$$x \longmapsto y \in \{0,1\}$$

My first-to-go

cross-entropy loss fun



loss

$$-\log p_{\hat{y}}$$

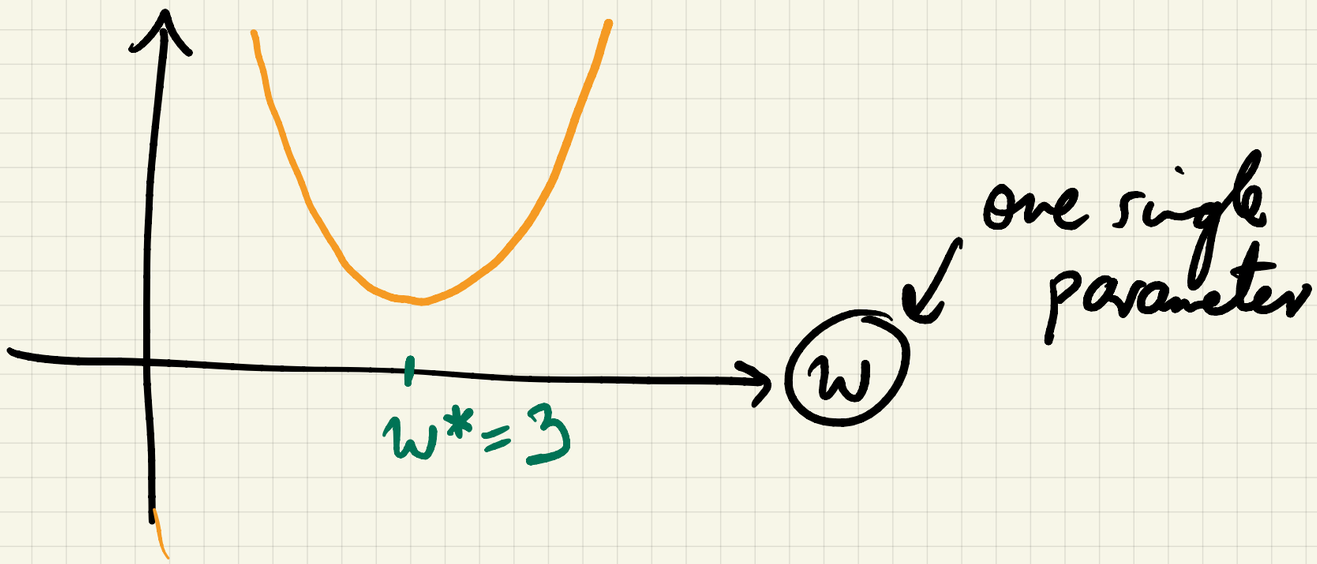
Regression

$$x \longmapsto y \in \mathbb{R}$$

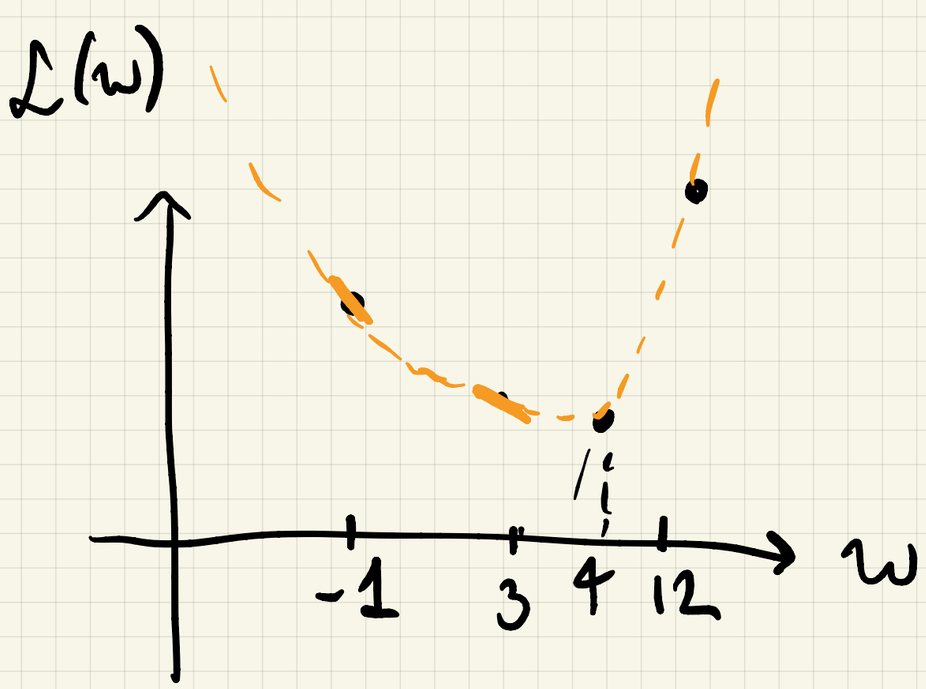
$$(y-\hat{y})^2$$

Squared error

How to optimize any loss function?



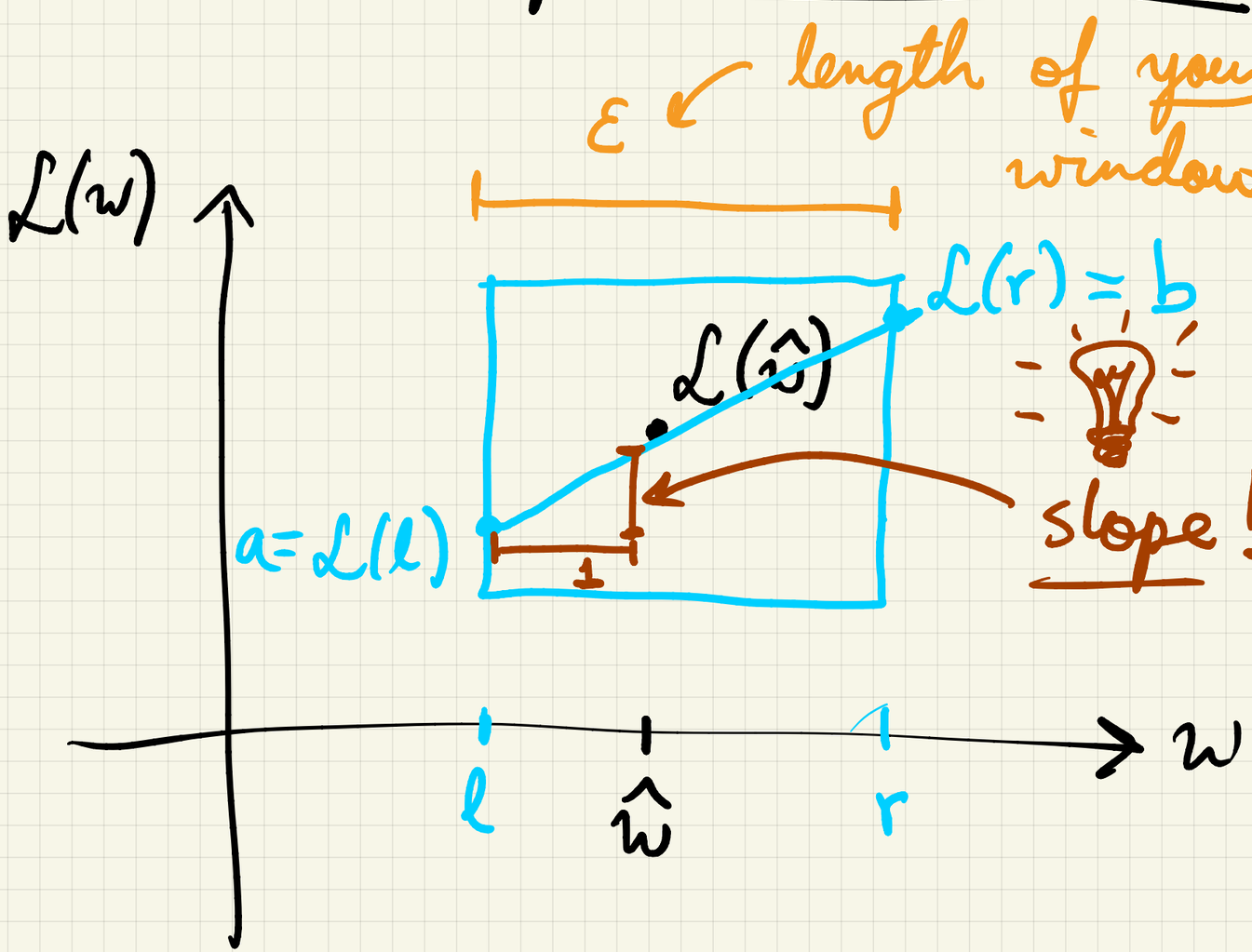
$$\mathcal{L}(w) = w^2 - 6w + 10$$



$$\mathcal{L}(w) = w^2 - 4w + 7$$

$\hat{w}$	$\mathcal{L}(\hat{w})$
12	103
-1	12
3	9

How to compute the window



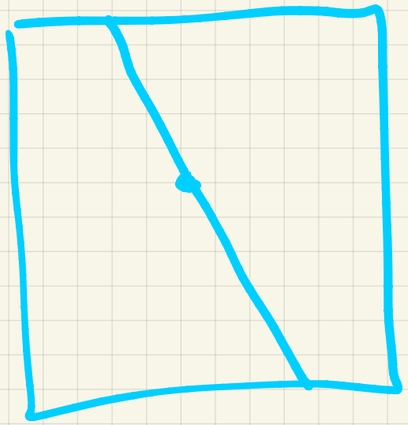
$$l = \hat{w} - \frac{\varepsilon}{2}$$

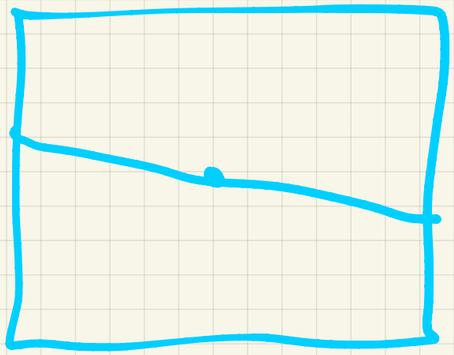
$$r = \hat{w} + \frac{\varepsilon}{2}$$

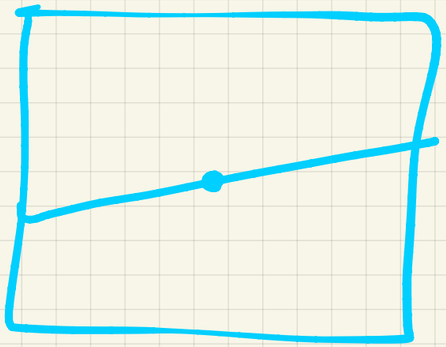
$$a = \mathcal{L}(l)$$

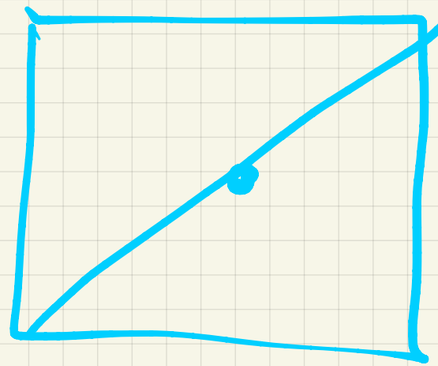
$$b = \mathcal{L}(r)$$

$$\frac{b-a}{\varepsilon} = \nabla_{\varepsilon} \mathcal{L}(\hat{w})$$









Desired update

slope

+

big negative

+

small negative

−

small positive

−

big positive

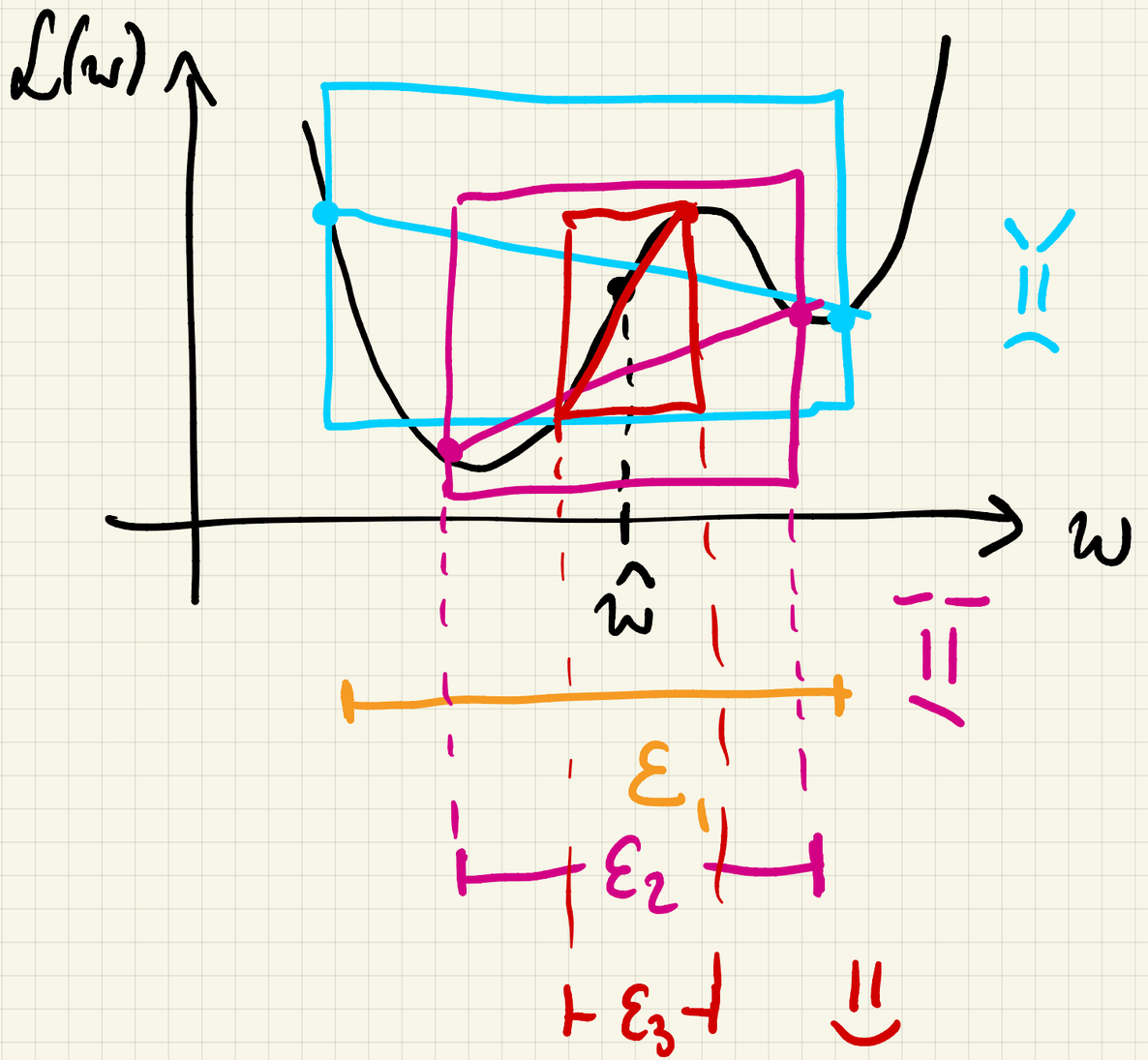


$$\hat{w} \leftarrow \hat{w} - \alpha \nabla_{\hat{w}} \mathcal{L}(\hat{w})$$

~~Slope~~

Gradient descent update

Warning!



$\nabla_{\varepsilon_1}\mathcal{L}(\hat{w})$ $\nabla_{\varepsilon_2}\mathcal{L}(\hat{w})$ $\nabla_{\varepsilon_3}\mathcal{L}(\hat{w})$ ...

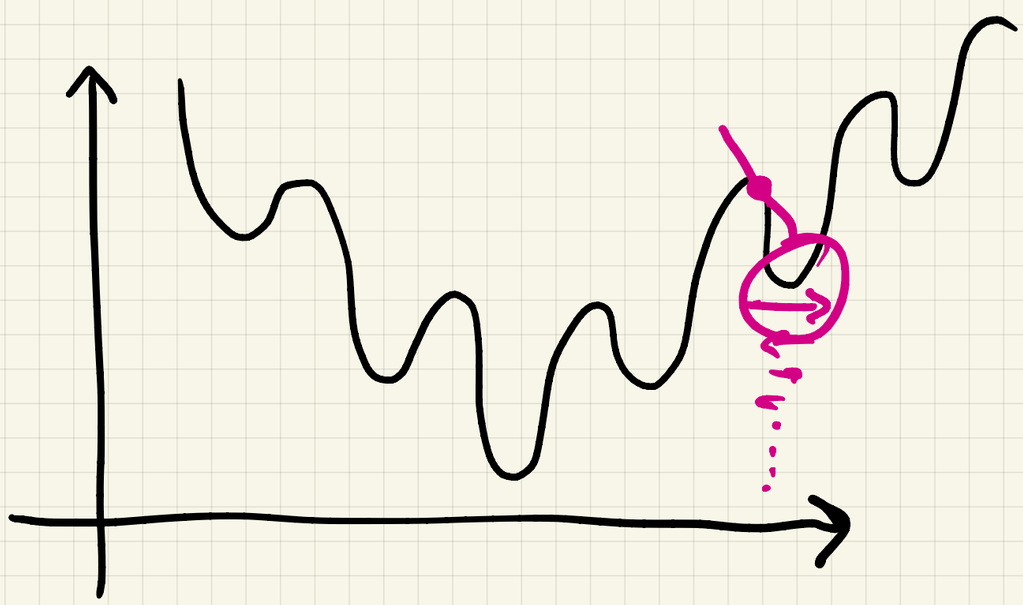
$\varepsilon_1 > \varepsilon_2 > \varepsilon_3 >$

THE Gradient

$\varepsilon = 0$

$\nabla\mathcal{L}(\hat{w})$

Gradient descent



do

$\hat{w} \leftarrow \hat{w} - \alpha \nabla \mathcal{L}(\hat{w})$

until $|\Delta| < \varepsilon$

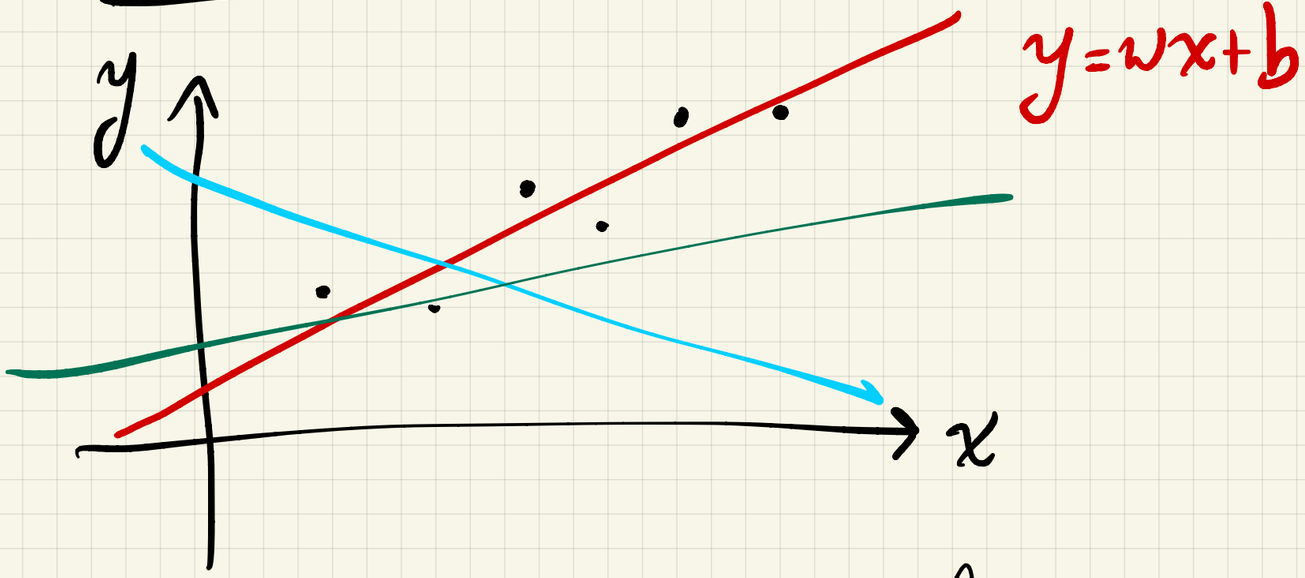
In practice $\hat{w} \in \mathbb{R}^d$

$$\hat{w} = (\hat{w}_1, \hat{w}_2, \ldots, \hat{w}_d)$$

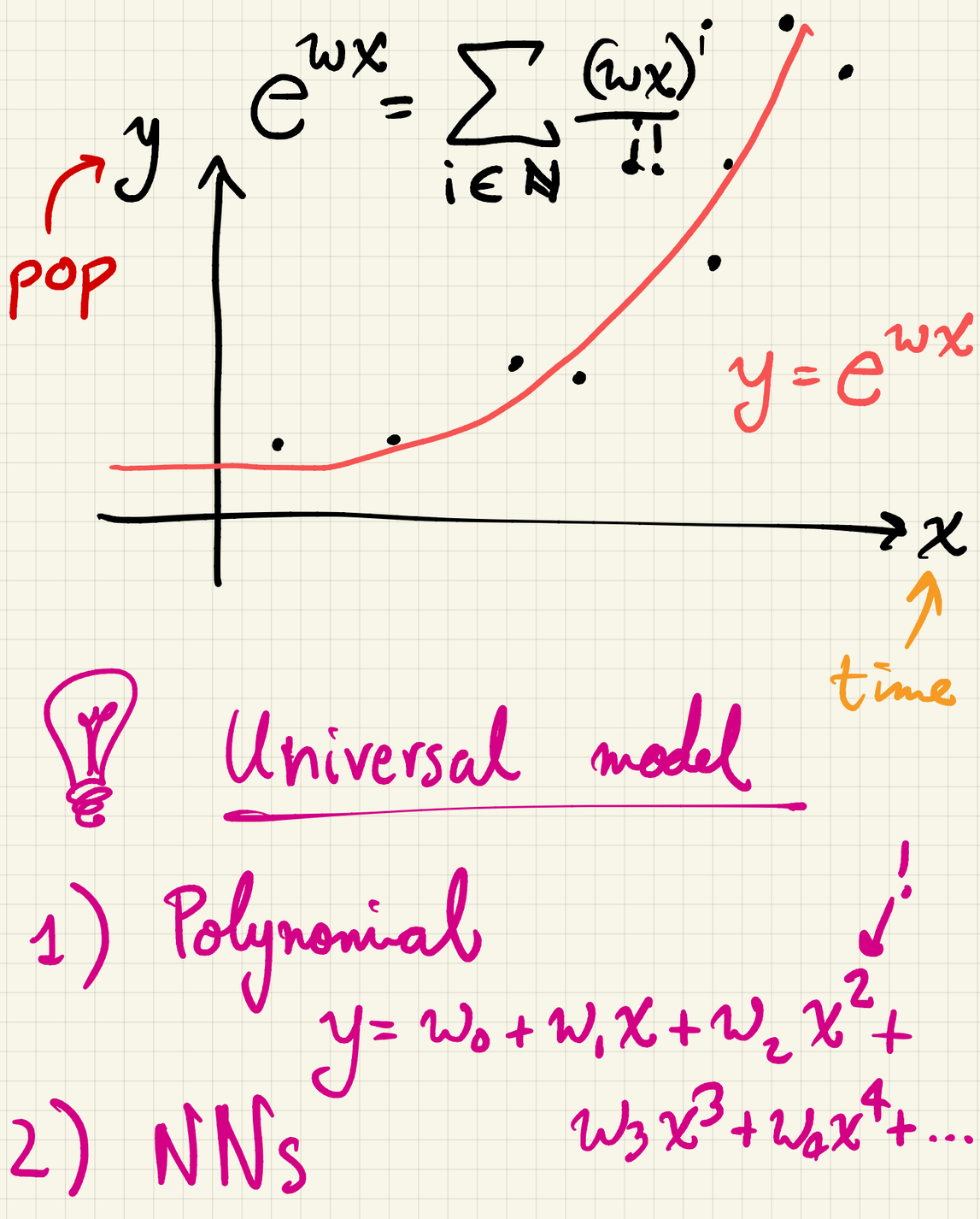
$\hat{w}_1$	l_1	r_1	a_1	b_1	slope$_1$	$\nabla_1 \mathcal{L}(\hat{w})$
$\hat{w}_2$	l_2	r_2	a_2	b_2	slope$_2$	$\nabla_2 \mathcal{L}(\hat{w})$
$\vdots$						
$\hat{w}_d$	l_d	r_d	a_d	b_d	slope$_d$	$\nabla_d \mathcal{L}(\hat{w})$

$$\nabla \mathcal{L}(\hat{w}) = (\nabla_1 \mathcal{L}(\hat{w}) \ \ldots \ \nabla_d \mathcal{L}(\hat{w}))$$

Models







Universal model

1) Polynomial

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + \dots$$

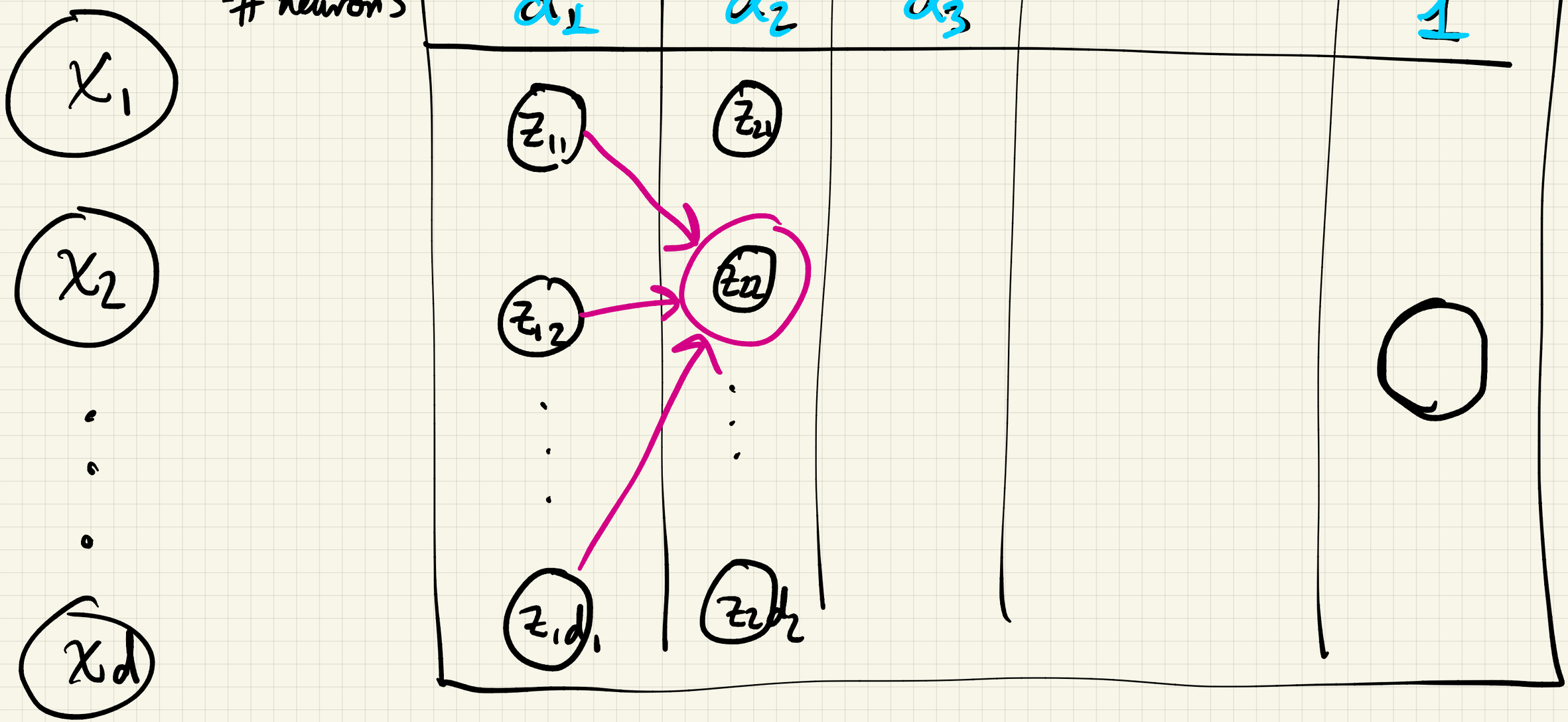
2) NNs

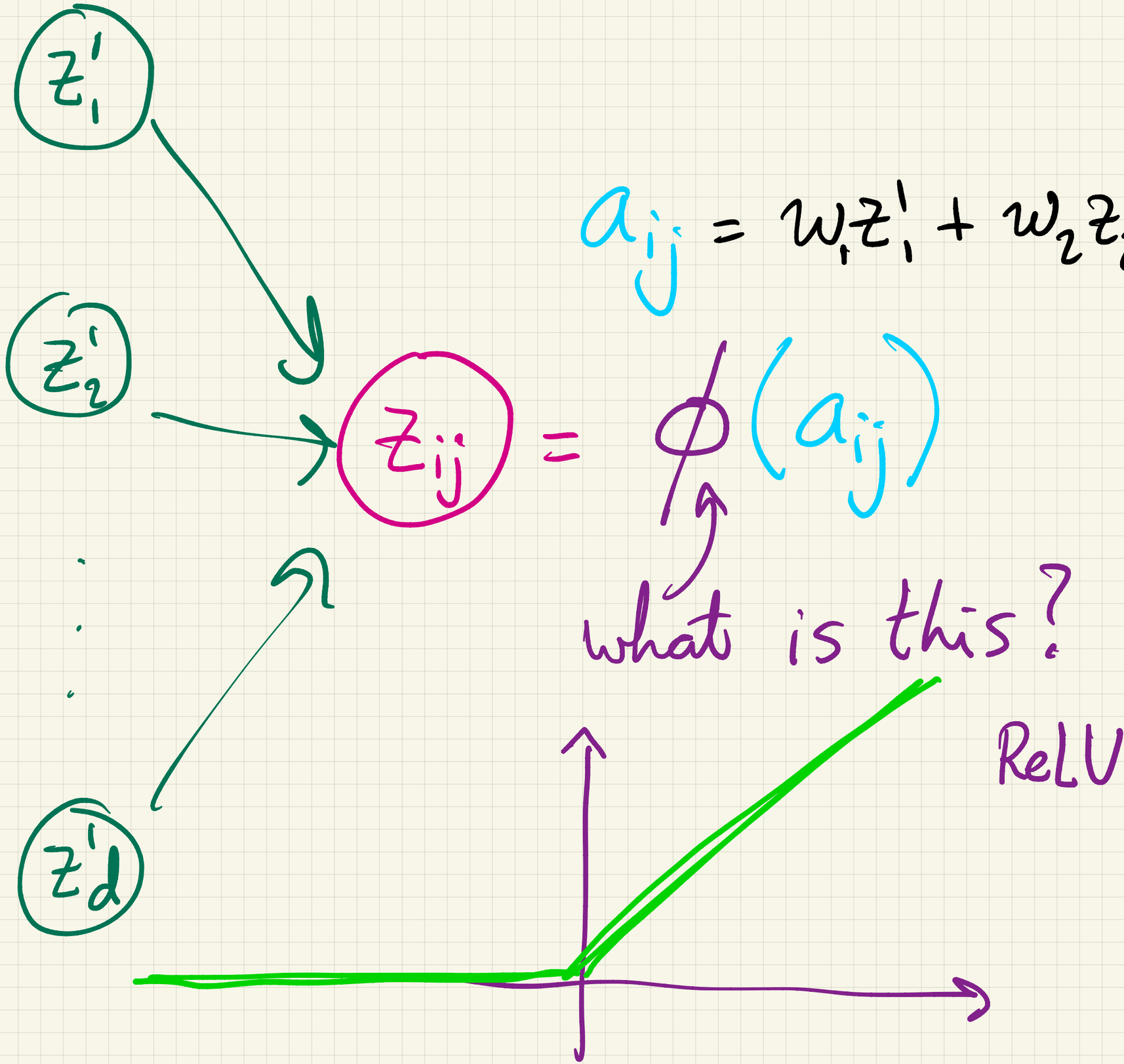
What is NN

$x = (x_1, \dots, x_d)$

layers L

	1	2	3	....	L
# neurons	d_1	d_2	d_3		1




z_1^1
z_2^1
z_d^1
z_{ij}
=
$\phi_g(a_{ij})$
$a_{ij} = w_1 z_1^1 + w_2 z_2^1 + \ldots + w_d^1$
what is this?
ReLU

Empirical risk minimization (ERM)

Suppose that a phenomenon of interest (e.g. house prices) is governed by an unknown **mathematical function** f^* that takes as input a list x of **features** (e.g. house measurements) and outputs a **target** y (e.g. house price).

Furthermore, suppose that

$$y = f^*(x) + \epsilon,$$

where ϵ is a random value that on average is 0. More precisely, $\epsilon \sim \mathcal{N}(0, \sigma^2)$, with unknown σ^2.

We don't have access to f^*. Instead, we have only a set of **observations** $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. How can we build an estimator $\hat{f}$ of f^*?

Empirical risk minimization (ERM)

Vapnik proposed **empirical risk minimization**

1. Collect **observations:** $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$**.**
2. Propose a **model** $\mathcal{H}$ comprising of all mathematical functions of a particular form. Examples:
   1. Linear functions: $y_i = f(x_i) = w\, x_i + b$.
   2. Polynomial functions: $y_i = f(x_i) = w_0 + w_1\, x_i + w_2 x_i^2 + \cdots + w_j x_i^j + \cdots$
   3. …
3. Compute the **empirical risk minimizer**

$$\hat{f} = argmin_{f \in \mathcal{H}} \sum_{i \leq n} \mathcal{L}\big(y_i, f(x_i)\big)\,.$$

Here, $\mathcal{L}$ is a function that measures differences between **targets** and **estimates.**

Vapnik elaborates in his book why this is the most reasonable approach. As of today, this remains the best strategy for estimating dependencies from data!

Example

- Suppose we observe only one house with area x_1 and price y_1.
- We **hypothesize** that $y = f(x) = wx$ for some unknown w. Mathematically, this means that we choose $\mathcal{H}$ as the set of all **linear functions.**
- We can define $\mathcal{L}(y, \hat{y}) = |y - \hat{y}|$.
- Our mission is then to find

$$\hat{f} = argmin_{f \in \mathcal{H}} \mathcal{L}\big(y_1, f(x_1)\big) = |y_1 - wx_1|.$$

Example

- Suppose we observe only one house with area x_1 and price y_1.
- We **hypothesize** that $y = wx$ for some unknown w. Mathematically, this means that we choose $\mathcal{H}$ as the set of all **linear functions.**
- We can define $\mathcal{L}(y, \hat{y}) = |y - \hat{y}|$.
- Our mission is then to find

$$\hat{f} = argmin_{f \in \mathcal{H}} \mathcal{L}\big(y_1, f(x_1)\big) = argmin_{w \in \mathbb{R}} |y_1 - wx_1|.$$

- This can be solved using the Robbins-Monro algorithm!

Robbins-Monro algorithm

Given x_1 and y_1, find w such that $|y_1 - wx_1| = 0$. That is, such that $y_1 = wx_1$.

1. Make a guess $\tilde{w}$
2. As long $|y_1 - \tilde{w}x_1|$ is unacceptably large, repeat the following
   1. If $y_1 < \tilde{w}x_1$, then decrease $\tilde{w}$.
   2. If $y_1 > \tilde{w}x_1$, then increase $\tilde{w}$.

A bit more advanced example

- Suppose we observe only one house with area x_1 and price y_1.
- We **hypothesize** that $y = wx + b$ for some unknown w and b. Mathematically, this means that we choose $\mathcal{H}$ as the set of all **linear functions.**
- We can define $\mathcal{L}(y, \hat{y}) = |y - \hat{y}|$.
- Our mission is then to find

$$\hat{f} = argmin_{f \in \mathcal{H}} \mathcal{L}\big(y_1, f(x_1)\big) = |y_1 - wx_1 - b|.$$

- This can be solved using the Robbins-Monro algorithm! The loop alternates between updating w and b.

How about multiple houses?

- In practice, we have many houses available: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$.
- Our goal then becomes

$$\hat{f} = argmin_{f \in \mathcal{H}} \sum_{i \leq n} \mathcal{L}\big(y_i, f(x_i)\big) = \sum_{i \leq n} |y_i - wx_i - b| \,.$$

- This can be solved using the alternating Robbins-Monro algorithm!

What if we also have the area, number of rooms, age, and other features of a house?

- In practice, the measurement $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$ is a vector.
- Our hypothesis changes to $y_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + b = w^\top x + b$ for some unknown $w = (w_1, \dots, w_d)$ and b.
- Our goal then becomes

$$\hat{f} = argmin_{f \in \mathcal{H}} \sum_{i \le n} \mathcal{L}\big(y_i, f(x_i)\big) = \sum_{i \le n} |y_i - w^\top x - b| \,.$$

- This can be solved using the altenating Robbins-Monro algorithm!

ERM today

Well… many popular modern ML applications follow the same principles!

	1982	Today
Learning strategy	Empirical risk minimization	Empirical risk minimization
Data	Dozens of low-dimensional observations	Thousands of high-dimensional observations
Loss function	Simple loss functions	Additive combinations of loss functions
Model / hypothesis	Linear models	Deep neural networks
Training algorithm	Robbins-Monro algorithm	Stochastic gradient descent

Use gradients

- The Robbins-Monro update rule uses only the difference between target and estimation.
- A more efficient update rule uses **the gradient of the loss function.**

Use neural networks instead of linear models

- Neural networks are **universal approximators.** They can approximate arbitrarily well any continuous function on a compact domain!

Use batches instead of the entire dataset

- At each iteration of the update loop, don't recompute the loss on the entire data; use only a "batch" (i.e., a subset of the dataset)
- Integrating all these improvements yields **stochastic gradient descent,** one of the most powerful modern algorithms for estimating dependencies on data.

Kahoot! contest

https://create.kahoot.it/details/46f33a94-f324-419d-8c59-5bd9248d8b4d